

Important Dates

Date	Note
09/28	DDL: A1 – project brief and pilot data
09/29	<i>Consultation 1 – collection gate</i>
10/20	Midterm review
10/22	Midterm exam
11/04	DDL: A2 – analysis-ready and EDA
11/05	<i>Consultation 2 – dataset gate</i>
12/08	<i>Consultation 3 – presentation gate</i>
12/10&15	Final presentation
12/17	DDL: A3 – final report

Schedule: First Half of Semester

Date	Topics
w1 09/01	Course introduction & house rules
09/03	Group project introduction
w2 09/08	Web scraping I
09/10	Web scraping II
09/15	Lab 1: scraper build
w3 09/17	Mock interview 1
09/22	Working with generative AI
w4 09/24	AI-assisted business analytics
09/29	<i>Consultation 1 – collection gate</i>
w5 10/01	National Day Holiday (no class)
10/06	
w6 10/08	Unstructured data analysis I
10/13	Unstructured data analysis II
w7 10/15	Unstructured data analysis III
10/20	Midterm review
w8 10/22	Midterm exam



温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 4701 W01 | Fall 2026

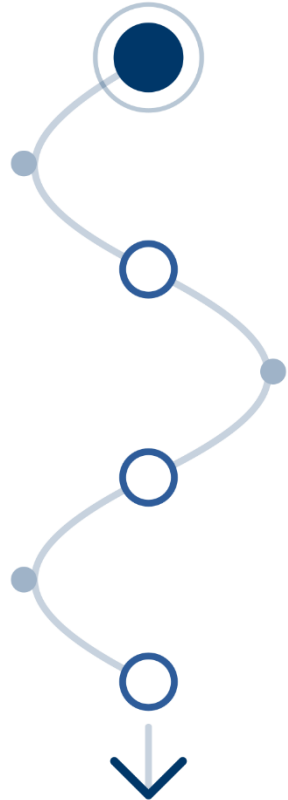
Integrative Application of Business Analytics

Web Scraping I: Concepts and Orientation

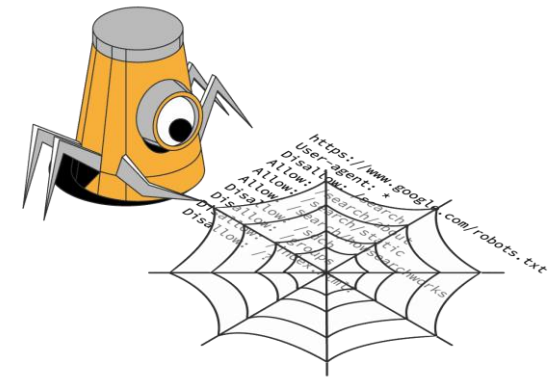
Dawei CHEN

Week 2-1 | Tuesday, 8 September 2026

Today



- 1 What is web scraping**
- 2 Why and when we need it**
- 3 How it works — by hand first, then by program**
- 4 Limitations and challenges**
- 5 A live demonstration**
- 6 Takeaways**



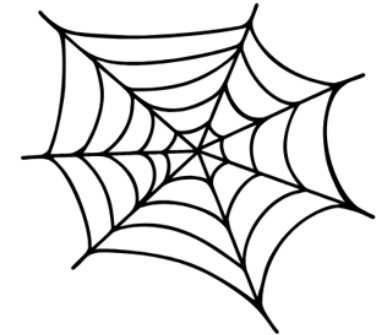
1. What Is Web Scraping

“Web scraping is data scraping used for extracting data from websites.”

Source: [Wikipedia, “Web scraping”](#).

Two actions, in this order

- **Fetch** – download the page exactly as a browser would
- **Extract** – take only the values you decided in advance to collect



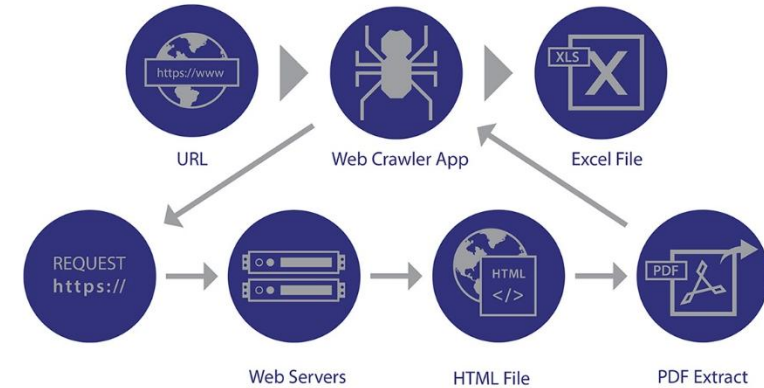
In a project,

- **Separated scripts** – collection, cleaning, visualization, modeling
- **Outputs of scraping** – raw responses as they arrived, a table with one row per record

1. What Is Web Scraping

In one sentence:

Imitate a browser opening a page, and take only the part of the data you want.



Three things follow:

- **Scope** – Whatever the server sends, you can get. What it never sends, you cannot.
- **Sequence** – “The part you want” has to be decided before you write code.
- **Standing** – You are copying what was served to you, not breaking in.

2. Why Web Scraping

Method	Answers questions about	Main cost
Survey	Stated attitudes and intentions	Design and response rate
Experiment	Cause and effect	Setup, ethics, validity
Administrative records	What happened inside a firm	Access and permission
Sensors and logs	Behavior over time	Instrumentation
Web data	What is publicly posted, at scale	Collection effort and care

*Scraping tells you **how to get web data**. It does not tell you **what your sample is**: which population, which period, which keywords*

2. Why Web Scraping

Four routes to **web data**, in order of preference

- 
- 1) **An existing dataset** – Someone has already collected and documented it.
 - 2) **An official API** – Structured, permitted, versioned, stable.
 - 3) **A documented feed, sitemap or archive** – Bulk access, no page parsing.
 - 4) **Scraping** – Last, because it breaks silently when someone edits their HTML.

- *Routes 1 to 3 are cheaper, more stable and easier to defend.*
- *Reaching for route 4 first is the most common and most expensive.*

2. Why Web Scraping

Scrape when

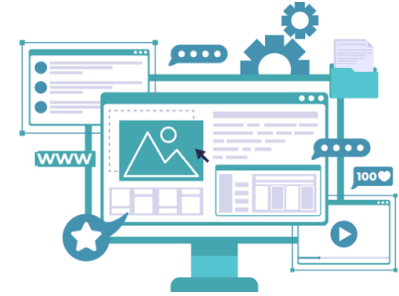
- No published dataset covers the question
- There is no API, or it omits the field you need
- The data is public and visible to any visitor
- You need it repeatedly, over time

Do not scrape when

- An API exists — use it
- The data sits behind a login or a paywall
- The page is mostly personal details of individuals
- Ten minutes of copying by hand would do



2. Why Web Scraping

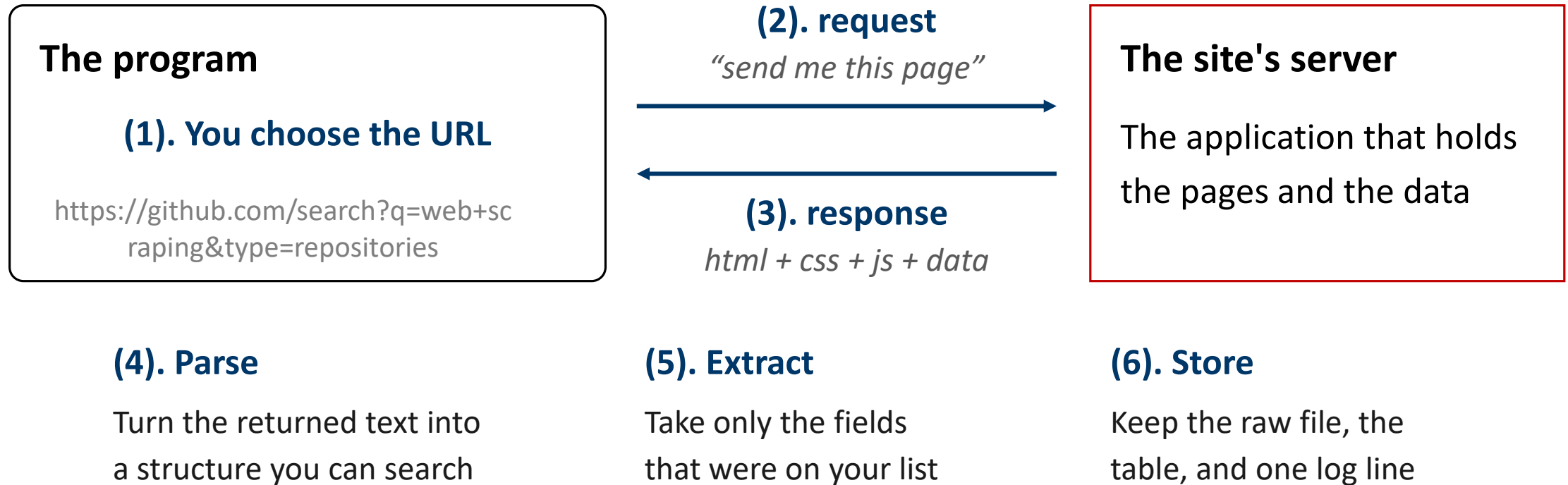


Where industry actually uses it

- **Retail and travel** – Competitors' prices and availability, checked daily.
- **Recruitment and workforce planning** – Which skills employers are asking for, and where.
- **Investment** – Job postings, reviews and listings as early indicators — sold as “alternative data”.
- **Brand and product teams** – Reviews and ratings across marketplaces.
- **Research** – Building a corpus that no vendor sells.

- *In most firms this is bought from a data vendor.*
- *Knowing how it is produced is what lets you judge whether to trust it.*

3. How Web Scraping Works



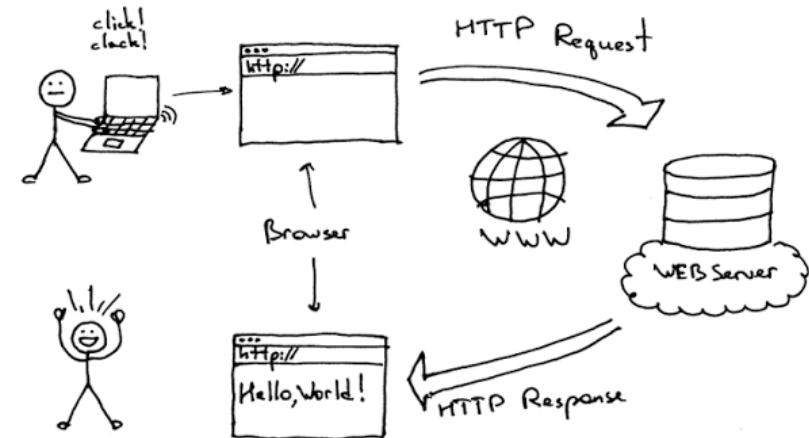
Steps 1 to 3 are exactly what your browser already does. Steps 4 to 6 are the key difference.

3. How Web Scraping Works

First, how would you *do it by hand*?

- **Task:** Collect BA job postings on LinkedIn from the last 90 days, across five cities.
- **What you have**
 - Twenty student assistants, a browser and Excel.
 - No code, no programming.

How would you organize it?



3. How Web Scraping Works



What the assistants would actually do

- 1). Choose the **source** – LinkedIn, not a search engine. Decide once, write it down.
- 2). Write the **keyword list** – “business analyst”, “data analyst”, “business intelligence”, and so on.
- 3). **Search, then filter** – City, date posted, experience level — and record which filters you clicked.
- 4). Open **each result** in turn, one posting at a time.
- 5). Copy the **fields into a row** – Title, company, city, salary, posted date, description, link.
- 6). **Repeat** – Next result, next page, next keyword, next city.

3. How Web Scraping Works

The spreadsheet you would end up with

job_title	company	city	salary	posted
Business Analyst	Alpha Retail	Shanghai	15–25K	2026-08-30
Data Analyst, Pricing	Beta Logistics	Hangzhou	not stated	2026-09-02

- **One row per posting.** Plus two columns not shown: the full description, and the source link.
- **Every column is a decision** you made at step 5 — and a column you forgot is one you cannot recover later.
- **Look at row two.** "Not stated" is a finding about the labor market, not a gap to fill in.

3. How Web Scraping Works

Now hand the same job to a program

Your manual step	What the program does	The tool/lib
1). Choose the source	The program does not decide this, you do	
2). Write the keyword list	Builds one URL per keyword	a Python list
3). Search, then filter	Sends the request with those parameters	requests
4). Open each result	Follows each result link	requests
5). Read the page, copy the fields	Finds each field by tag and class	BeautifulSoup
6). Repeat and save	Loops the pages, then writes the file	pandas

Nothing new happens. Every line of code replaces something you would otherwise have done with a mouse.

3. How Web Scraping Works

What each library is for

Library	The manual equivalent	What it does
requests	Typing the address, pressing enter	Fetches the page as text
BeautifulSoup	Reading the page, finding the salary	Searches that text by tag and class
Selenium	Opening a browser, clicking, scrolling	Drives a real browser when the page needs one
pandas	The spreadsheet itself	Holds the rows, writes the file

You already know the last row. The first three are the new part.

3. How Web Scraping Works

Where do the fields sit on the page?

- Step 5 by hand is “look at it”. For a program it is “find the label”.

```
<div class="job-card">
  <h2 class="title">Data Analyst</h2>
  <span class="company">Alpha Retail</span>
  <span class="salary">15-25K / month</span>
</div>
```

- **tag** — what kind of thing it is
- **class** — which group it belongs to: this is the label you search for
- **text** — the value that goes in your spreadsheet cell



[HTML tutorial](#)

3. How Web Scraping Works

Why do some pages need a browser?

	Static page	Dynamic page
<i>Where the data is</i>	In the first response	Added afterwards by JavaScript
<i>The test</i>	View Source, search your text — it is there	View Source, search — missing, though visible on screen
<i>What works</i>	requests + BeautifulSoup	Selenium , or the page's own hidden API

LinkedIn is dynamic and needs a login, which make it difficult to scraping data from it.

4. Limitations and Challenges



One row is easy. Five thousand is not.

- A person collects one row without thinking.
- A program collecting five thousand meets **problems a person never notices**.
 - **You would notice a missing salary.** The program writes an empty cell and carries on.
 - **You would wait for the page to finish loading.** The program reads whatever arrived first.
 - **You would stop after twenty pages.** The program asks for five thousand in a minute.

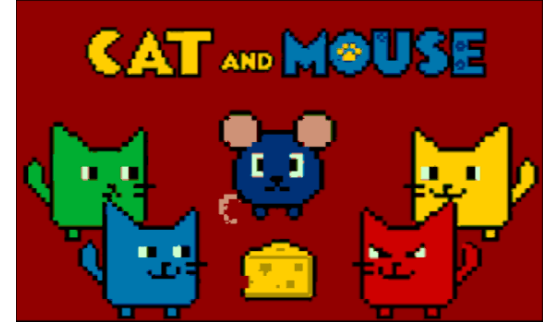
None of these are programming problems. They are all consequences of removing the human who was quietly checking as they went.

4. Limitations and Challenges



Challenge	By hand	For the program
Scale and pagination	You click "next" until it ends	It must know when to stop, or re-reads page one forever
The page loads itself	You wait a second, then read	It reads an empty shell unless it waits or calls the API
Being blocked	Nobody stops you reading twenty pages	Thousands of requests look like an attack, and get refused
Silent breakage	You would spot a wrong salary at once	It fills the column with blanks and reports success

4. Limitations and Challenges



What scraping cannot fix

- Fields that are simply **absent** — no library can produce a salary the employer never posted.
- Values the site **estimated** rather than the employer stated.
- The same posting listed **three times** by three different agencies.
- A page that changes **next week** and breaks your selector silently.
- Anything **behind a login**, a paywall, or a block.

Every one of these belongs in your methods section as a finding, not hidden as a defect.

Rules of Collection

- 1). Read robots.txt before the first request
- 2). One request per second, no concurrency
- 3). Identify yourself in the User-Agent
- 4). Never log in, and never scrape behind a login
- 5). Never bypass a block, a limit or a CAPTCHA
- 6). Collect the posting, not the poster
- 7). Keep the raw response, not only the table
- 8). Log every run: date, URL, parameters, rows
- 9). If an API exists, use it
- 10). If in doubt, ask before collecting, not after

Rule 6 has no exceptions: collect the posting, never the poster — no names, photographs, contact details or personal profiles, whatever the site permits.

5. A Live Demo



<https://movie.douban.com/>

Before the prompt, the robots file: movie.douban.com/robots.txt

- **Search paths are disallowed** — /search, /subject_search and several others.
- The top-250 page is not among them, so it is not excluded.
- A crawl delay of five seconds appears, commented out. We honour it anyway.

Rule 1, applied in ninety seconds, in front of you. This is the step everyone skips.

Next: one prompt in VS Code, and a notebook that collects the top 100 films.

5. A Live Demo



<https://movie.douban.com/>

One prompt, one notebook

```
"In VS Code, create a Jupyter notebook that collects the
Douban Top 250 films – first four pages only – recording
title, year, rating, number of ratings and the source URL,
with a five-second delay between pages. Return a pandas
DataFrame and save it to CSV with the collection date."
```

- **Source, fields, limit, rate, output** – The same five decisions the assistants made by hand.
- A vague prompt produces plausible code and confidently wrong numbers.

5. A Live Demo

The program ran $\neq$ Get the desirable data

- 1) Did you get one hundred rows, or twenty-five? Check against pages $\times$ films per page.
- 2) Is the rating a number, or text that merely looks like one?
- 3) Did any film lose a field — and did its row survive, or vanish?
- 4) Does every row carry the source URL and the collection date?



Thursday: *we collect from a real site, end to end.*

Takeaways

1. Scraping imitates a browser and keeps the part you asked for. Nothing more.
2. It is one route to web data, and the last of the four routes.
3. Decide by hand first. Source, keywords, filters, fields — then automate.
4. The hard problems arrive with volume, not with the first row.

AI Use Disclosure

What the AI assistant did

- Drafted wording from an outline I specified
- Proposed a slide sequence, which I reordered twice
- Generated the file from our course template

What I did

- Set the structure, the argument and the examples
- Checked the sources, including the robots file
- Corrected its errors, including a slide that miscounted its own code example

The same standard applies to your project: name the tool, keep the judgement, and own the errors it makes on your behalf.

Q&A

Dawei CHEN | CBPM B214 | dchen@kean.edu

Office hours: Tue 16:00 – 17:15 | Wed 15:00 – 16:00 | Thu 14:15 – 17:00

Next session: Thursday 10 September, 13:00 — Web Scraping II: a real notebook, end to end